Project Report: TrekFlow – Trekking Management System

Student Details

- **Name:** Manish Kumar
- **Roll Number:** 24f3004117
- **Email:** 24f3004117@ds.study.iitm.ac.in

- **About Me:** I am a student in the IIT Madras BS Degree Diploma program with a strong interest in web application development, backend server configuration, and deploying scalable tools. I enjoy building structured platforms that solve real-world operational challenges while focusing on minimalist, modern UI/UX design.

1. Problem Statement

Managing outdoor trekking trips has traditionally been messy, relying on scattered spreadsheets and decentralized communication. This creates headaches for everyone involved:

- **Trekkers:** Struggle to find trips that match their skill level or location and have a hard time keeping track of their bookings.
- **Field Staff:** Lack a central way to see real-time updates on route progress, manage group lists, or update the number of available seats.
- **Administrators:** Face challenges in securing user data, managing staff assignments, and handling platform-wide issues like last-minute cancellations.

TrekFlow solves these problems with an all-in-one platform that makes booking and management simple, transparent, and secure.

2. Solution Approach

The application is built using the Python Flask framework, following a standard Model-View-Controller (MVC) structure to keep the code organized and manageable:

- **Database Security:** We use SQLAlchemy to keep data organized and secure. User passwords are protected with industry-standard encryption (bcrypt).
- **Data Validation:** We use Flask-WTF to ensure that every piece of information entered—from emails to dates—is valid before it reaches our database.
- **Access Control:** To keep the platform safe, we use Flask-Login to manage user sessions. This ensures that users only see and edit information they are authorized to access.

3. AI/LLM Declaration

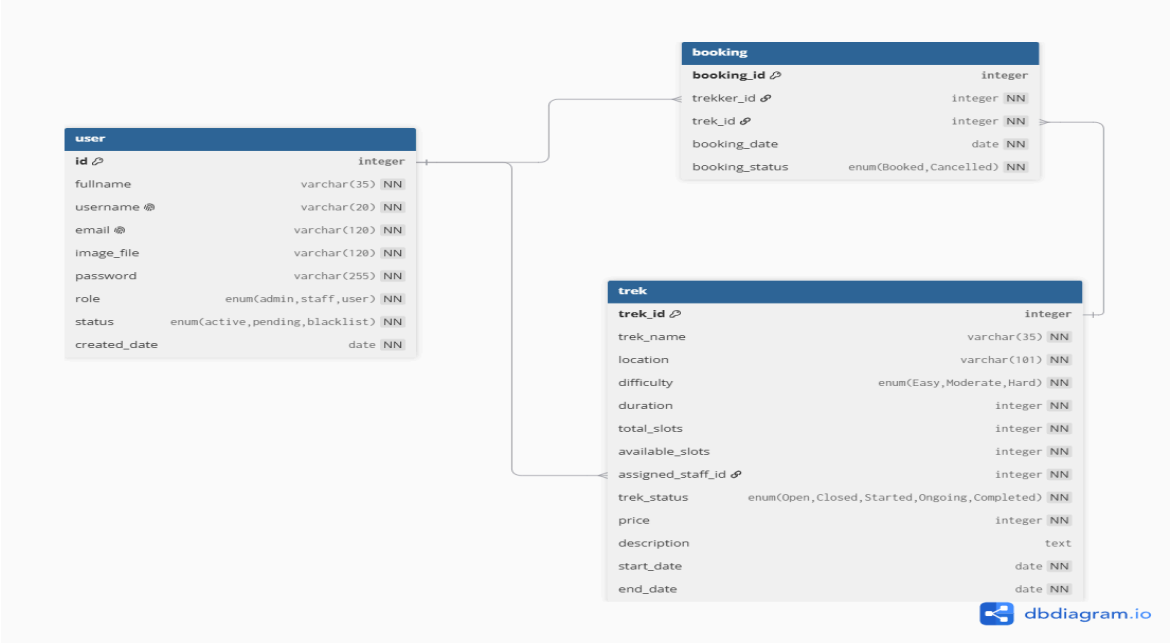
During the development of TrekFlow, AI tools such as ChatGPT and Google Gemini were used primarily as learning and development assistants. They were used to understand programming concepts, discuss implementation approaches, troubleshoot errors, refine SQL queries, improve form validation, and obtain UI/UX suggestions. All final design decisions, coding, and integration were performed manually.

- **Overall AI/LLM usage is around 15%**

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core WSGI backend web application framework
Flask-SQLAlchemy	Object Relational Mapper (ORM) for secure database interactions
Flask-WTF	Secure frontend form binding and data validation
Flask-Bcrypt	Cryptographic engine for hashing and securing passwords
Flask-Login	User authentication and multi-tier session management
Pillow (PIL)	Image processing for user profile picture uploads
Bootstrap 5	Frontend styling, responsive design, and UI components
SQLite	Lightweight local database (site.db) for data storage

5. Database Schema / ER Diagram



Tables:

- **User:** Stores user profile details and roles (id, fullname, username, email, password, role, status).
- **Trek:** Stores expedition details (trek_id, trek_name, location, difficulty, duration, available_slots, trek_status, assigned_staff_id).
- **Booking:** Logs user reservations (booking_id, trekker_id, trek_id, booking_status, booking_date).

Relationships:

- **One-to-Many:** User → Trek (Staff assignments)
- **One-to-Many:** User → Booking (User reservations)
- **One-to-Many:** Trek → Booking (Trek reservations)

6. Web Application Routes

Route	Methods	Function Name	Description
/	GET	home	Renders the primary landing homepage.
/login	GET, POST	login	Validates credentials and redirects users by role.
/register	GET, POST	register	Captures registration and sets status.
/logout	GET	logout	Ends active session.
/treks	GET, POST	treks	Displays Open treks with filters.

Route	Methods	Function Name	Description
/treks/<int:trek_id>/book	POST	book_trek	Processes ticket reservations.
/booking	GET	mybooking	Displays user bookings.
/booking/<int:booking_id>/details	GET	viewbooking	Displays booking details.
/history	GET	history	Renders completed trek history.
/profile	GET, POST	profile	Manages trekker account updates.
/staffdashboard	GET	staffdashboard	Primary staff view.
/staffdashboard/treks	GET	stafftreks	List of assigned treks.
/staffdashboard/<int:trek_id>/manage	GET, POST	staffmanagetreks	Updates trek constraints and state.
/staffdashboard/profile	GET, POST	staffprofile	Staff-specific profile management.
/admindashboard	GET, POST	admindashboard	Global metrics portal.
/admindashboard/staff	GET, POST	admin_staff	Staff list management.
/admindashboard/staff/<int:user_id>/approve	POST	approve_staff	Approves pending staff.
/admindashboard/staff/<int:user_id>/blacklist	POST	blacklist_staff	Revokes staff access.
/admindashboard/newtrek	GET, POST	admin_trek	Form to construct new trek.
/admindashboard/newtrek/save	POST	adding_trek	Commits new trek to database.
/admindashboard/treks	GET	view_treks	Aggregate trek monitoring.
/admindashboard/treks/<int:trek_id>/edit	GET, POST	edit_treks	Edits trek details.
/admindashboard/treks/<int:trek_id>/delete	POST	delete_trek	Closes trek state.
/admindashboard/users	GET, POST	admin_users	User account review roster.
/admindashboard/users/<int:user_id>/activate	POST	activate_user	Reinstates user privileges.

Route	Methods	Function Name	Description
/admindashboard/users/<int:user_id>/blacklist	POST	blacklist_user	Suspends booking privileges.
/admindashboard/bookings	GET	booking_admin	Global booking ledger.
/admindashboard/bookings/<int:booking_id>/cancel	POST	admin_cancel_booking	Cancels booking and refunds slot.

7. Architecture and Features

Architecture Overview:

- run.py – Main application execution script.
- src/trekflow/model.py – Database models defined using SQLAlchemy.
- src/trekflow/routes.py – Core application logic and routing.
- src/trekflow/forms.py – Flask-WTF form classes and custom validators.
- src/trekflow/templates/ – Jinja2 HTML templates segregated by role.

Implemented Features:

- Strict role-based access control (Trekker, Staff, Admin).
- Dynamic trek booking system with real-time slot tracking.
- Admin management suite for personnel approval and blacklisting.
- Staff dashboards for updating trek lifecycles (Open, Ongoing, Completed).
- Secure password hashing and protected routing middleware.

8. Video Presentation

- **Drive Link:** Presentation Video